ChatGPT

C++ 趣味学习网站 0→1 可执行方案深度研究报告

执行摘要

本报告给出一份面向 **2026-03-07（新加坡时区）**即可启动实施的 0→1 可落地方案：以“**学习地图 + 互动课堂 + 闯关评测 + 成长系统**”为产品闭环，技术上采用“**主站（课程/用户/进度/题库）模块化单体 + 独立判题集群**”的架构，前端以 **Monaco Editor** 作为核心编辑器（PC 优先，移动端以浏览/打卡为主），互动演示以 **WebAssembly（Emscripten 工具链）**承载关键概念可视化，判题层优先选用 **Judge0 CE 自托管**以快速获得可靠的多语言编译执行能力与资源限制能力，同时配合容器安全（rootless、seccomp、AppArmor、NetworkPolicy）实现对不可信代码执行的隔离与审计。Monaco 官方明确指出其不支持移动浏览器/移动 Web 框架，因此 MVP 的“写代码体验”应明确以桌面浏览器为默认场景。[1]

判题接口建议采用异步（wait=false）：Judge0 API 文档明确提示 **不推荐 wait=true（不易扩展）**，因此应以提交后轮询/事件推送方式获取结果，并把“实时反馈”留给 SSE/WebSocket 通道实现。[2] 资源限制（CPU/墙钟/内存/进程数/文件大小/网络等）可直接使用 Judge0 提供的配置与限额体系落地。[3]

数据库以 **PostgreSQL** 为主（课程内容块与配置可用 jsonb），jsonb 在 PostgreSQL 中以分解的二进制格式存储、通常处理更快且支持索引；而 GIN 索引是面向复合值/倒排场景的关键能力，适合为 jsonb 与标签检索提供性能保障。[4] 运维层建议以 GitHub Actions 做 CI/CD（YAML 工作流），以 Prometheus + Grafana + OpenTelemetry 建立指标/告警/链路追踪闭环，并通过 pg_dump 与 pg_basebackup 组合实现“逻辑备份 + 基础备份/PITR”策略。[5]

执行摘要后可立即开干的五项任务清单

任务	Owner 建议	产出物	完成标准
明确 MVP 学习路径与关卡结构（课程大纲 + 题型配比 + 成就体系骨架）	产品/教研	课程地图（20–30 节）+ 题库清单（100–150题）+ 成就清单（30个）	可直接录入后台；每节课明确目标、示例、练习、验收点
搭建判题 PoC（Judge0 CE 自托管）并打通提交链路	后端/平台	可运行的 Judge0 环境 + 平台提交到判题回写 Demo	完成 “提交→返回 token→查询结果→写库” 全链路；禁止网络/限制资源
设定数据库与 API 契约（OpenAPI/接口清单）并生成脚手架	后端	ER 草图 + 迁移脚本 + API 草案	核心表可迁移；接口可 mock；鉴权/限流策略可跑通
前端编辑器与运行面板原型（Monaco + SSE 更新）	前端	Lesson 页原型：讲解+编辑器+输出+状态	能编辑/运行样例；SSE 推送运行状态；移动端降级提示
建立 CI/CD 与可观测性最小闭环（构建、部署、监控、备份）	DevOps/平台	Actions 工作流 + Prometheus/Grafana + 备份计划	staging 自动部署；关键指标与告警可用；备份可恢复演练一次

范围与关键假设

为让成本、容量、里程碑估算可执行，本报告采用以下可调整假设（若你的目标用户/并发不同，可按本报告提供的扩容与成本公式替换参数）：

假设项	默认假设	影响的设计点
首发语言/题库	仅 C++（建议 C++17）	判题镜像、教材内容、题库标签、模板代码
首发终端	PC Web 为主；移动端以阅读/打卡为主	Monaco 移动端不支持的产品策略与降级路径 [1]
用户规模（首月）	注册 5,000；DAU 500	课程/题库规模、缓存与索引、判题并发
峰值在线写代码人数	100 并发编辑；峰值提交 10 req/s	判题队列长度、Worker 数、SSE 通道数
代码运行模型	“运行”=开放沙箱（可输入 stdin）；“提交”=题目测试用例判定	题目必须有隐藏用例；运行/提交分级限流
课程结构化内容	Lesson 由“内容块（block）”组成，落库 jsonb	PostgreSQL jsonb + GIN 索引策略 [6]
判题选型	MVP 采用 Judge0 CE 自托管；后续可替换自研/Isolate 直连	Judge0 提供资源限制配置与异步提交建议 [7]

非 MVP 范围（建议延后）：社区论坛、多人实时协作 IDE、AI 自动出题、复杂赛事系统、多语言国际化（可在第 2 阶段加入）。

MVP 功能清单与优先级

功能优先级总表

表格按 **必选/次选/可选**分组，每项包含实现要点与验收标准（建议你把这张表直接变成 Jira/Epic）。

分组	功能模块	功能点	实现要点	验收标准（可测试）
必选	账号与鉴权	邮箱注册/登录、JWT+刷新、退出	密码哈希按 OWASP 建议（如 Argon2/bcrypt/scrypt）并支持强密码长度；刷新令牌放 httpOnly Cookie；登录风控（限流、失败计数） [8]	注册/登录/刷新/退出全流程通过；错误次数超限触发限流；密码不明文存储
必选	学习路径	学习地图（章节解锁、先修关系）	Path→Node→Lesson；节点解锁依据“完成度/通关题”；支持 A/B 课程路线	新用户可从 0 开始；完成节点后自动解锁后续；先修未完成不可跳过

分组	功能模块	功能点	实现要点	验收标准（可测试）
必选	课程内容	Lesson 内容块渲染（文本/代码/小测/运行器）	课程内容存 jsonb；前端按 block schema 渲染；版本化发布	20–30 节可发布；每节课至少 1 个互动或练习；回滚到旧版本可用
必选	在线编辑器	Monaco Editor 集成（PC 优先）	Monaco 是 VS Code 核心编辑器；官方声明不支持移动端，移动端提供降级 UI（只读/跳转 App/提示） [1]	PC：编辑/高亮/缩进正常；移动端：明确提示并可浏览内容
必选	运行与提交	“运行代码”与“提交判题”两种入口	运行：快速反馈；提交：走题目测试；二者都写 submission 记录	运行在 5s 内返回结果（或提示排队）；提交返回通过/失败与关键用例反馈
必选	判题与安全	Judge0 CE 自托管 + 资源限制 + 禁网	Judge0 支持时间/内存/进程数/文件大小/网络等配置；API 明确不推荐 wait=true；使用 wait=false+异步回传 [9]	任意恶意代码无法访问内网/公网；超时/超内存可被正确终止并记录
必选	题库系统	题目列表/详情/标签/难度/模板代码	题目支持隐藏用例；模板代码按 C++17；支持多文件（后续项目题）	100–150 题可用；题目可按标签筛选；提交记录可追溯
必选	进度与成长	学习进度、经验值、连续学习、徽章	Redis Sorted Set 用于排行榜；经验值日志落库；连续学习按日历计算 [10]	完成 Lesson/题目后进度与 XP 实时更新；排行榜可分页且稳定
必选	后台管理	课程/题目/用例/发布管理	最小 CMS：内容块编辑、题目编辑、用例上传、发布审批	教研可独立发布课程与题目；发布后前台可见；修改有审计
次选	趣味互动	WebAssembly 概念演示组件（2–3 个）	Emscripten 是面向 WebAssembly 的完整编译工具链；可用 Embind 或 ccall/cwrap 与 JS 交互 [11]	至少 2 个可视化：栈/堆与引用/指针；能随代码变化更新演示
次选	实时体验	SSE 推送提交状态（必要时 WebSocket）	SSE 是服务器→客户端单向流；客户端 EventSource 接收事件 [12]	提交后页面不刷新即可看到状态变化；断线可自动重连

分组	功能模块	功能点	实现要点	验收标准（可测试）
次选	推荐系统	基于标签的练习推荐	“本课标签 → 题库相同标签”召回；按掌握度排序	学完一课自动推荐3–5题；点击转化可统计
次选	合规与风控	代码抄袭检测（离线批处理）	可接入 Moss（Stanford）或 JPlag（本地计算不上传代码）[13]	对同题提交可生成相似度报告；支持人工复核与白名单
可选	社交化	班级/小组、PK、赛季活动	活动配置化；排行榜按赛季分桶	赛季切换不影响历史；活动可灰度发布
可选	多终端	移动端编辑器替代（CodeMirror等）	因 Monaco 移动端不支持，移动端可换轻量编辑器或 App	移动端可完成基础练习（非必需）

数据库表设计

选型与建模原则

主库建议使用 PostgreSQL：课程内容块与配置类数据以 jsonb 保存，既保留灵活性，又可通过索引获得可用的查询性能；PostgreSQL 文档明确指出 jsonb 以分解的二进制格式存储、通常处理更快且支持索引，是大多数应用的优先选择。[14]
对 jsonb 字段，建议用 GIN（倒排）索引；PostgreSQL 文档说明 GIN 适合在“复合值中查找元素值”的查询模式。[15] 另外，jsonb_path_ops 在特定操作符（@>、@?、@@）上有明显性能优势，可针对内容块/配置查询使用。[16]

核心表清单

领域	表名	主键	用途
用户	users	uuid	用户主体、状态、注册来源
鉴权	user_credentials	uuid	密码哈希/盐、策略、风控字段
会话	refresh_tokens	uuid	刷新令牌、设备信息、吊销
内容	courses	uuid	课程元数据、封面、难度
内容	lessons	uuid	章节元数据、先修关系、发布版本
内容	lesson_blocks	uuid	章节内容块（jsonb）、可交互组件配置
题库	problems	uuid	题目、标签、模板代码、判题配置

领域	表名	主键	用途
题库	test_cases	uuid	测试用例（可含隐藏/分组/权重）
评测	submissions	uuid	运行/提交记录、Judge0 token、状态
评测	submission_results	uuid	编译输出、运行输出、资源消耗汇总
评测	submission_test_results	uuid	每个测试点结果（可加密/脱敏）
学习进度	user_lesson_progress	(user_id, lesson_id)	章节完成度、最后访问、得分
成长	xp_logs	uuid	经验值流水（可审计）
成长	achievements / user_achievements	uuid	徽章定义与用户获得记录
系统	audit_events	uuid	管理后台操作、关键安全事件审计

下面给出关键表的字段、类型、索引建议与示例记录（示例以 PostgreSQL 为准）。

users

字段	类型	约束	说明	索引建议
id	uuid	PK	用户 ID	主键
email	text	UNIQUE NOT NULL	邮箱	UNIQUE(email)
nickname	text	NOT NULL	昵称	idx_users_nickname（可选）
avatar_url	text		头像	
role	smallint	NOT NULL DEFAULT 1	1=普通用户 9=管理员	idx_users_role
status	smallint	NOT NULL DEFAULT 1	1=正常 2=封禁	idx_users_status
created_at	timestamptz	NOT NULL	创建时间	idx_users_created_at
last_login_at	timestamptz		最后登录	idx_users_last_login_at

示例记录：

```
{
 "id": "8b2e8b9a-3c3a-4b0f-aac7-0b0f1bd8d4a1",
 "email": "allen@example.com",
 "nickname": "Allen",
 "role": 1,
 "status": 1,
```

```
  "created_at": "2026-03-07T08:00:00Z"
}
```

user_credentials

字段	类型	约束	说明	索引建议
id	uuid	PK	记录 ID	主键
user_id	uuid	FK users(id) UNIQUE	每个用户一条（MVP）	UNIQUE(user_id)
password_hash	text	NOT NULL	密码哈希（Argon2/bcrypt 等）	
password_algo	text	NOT NULL	例如 argon2id	
failed_login_count	int	NOT NULL DEFAULT 0	连续失败计数	
locked_until	timestamptz		锁定到期	idx_locked_until
updated_at	timestamptz	NOT NULL	更新	

说明：密码存储与认证策略建议遵循 OWASP Password Storage 指南（核心思想：即使 DB 泄露也要尽量保护密码）。[17]

refresh_tokens

字段	类型	约束	说明	索引建议
id	uuid	PK		
user_id	uuid	FK users(id)		idx_refresh_user
token_hash	text	NOT NULL	刷新令牌哈希（不存明文）	UNIQUE(token_hash)
device_id	text		设备标识	idx_refresh_device
ip	inet		登录 IP	idx_refresh_ip（可选）
user_agent	text		UA	
expires_at	timestamptz	NOT NULL	过期时间	idx_refresh_expires
revoked_at	timestamptz		吊销时间	idx_refresh_revoked

courses

字段	类型	约束	说明	索引建议
id	uuid	PK		
slug	text	UNIQUE NOT NULL	URL 友好	UNIQUE(slug)

字段	类型	约束	说明	索引建议
title	text	NOT NULL	标题	idx_course_title（全文检索可选）
summary	text		简介	
difficulty	smallint	NOT NULL	1–5	idx_course_diff
cover_url	text		封面	
tags	text[]		标签数组	GIN(tags)
status	smallint	NOT NULL DEFAULT 1	1=草稿 2=发布	idx_course_status
published_at	timestamptz		发布时间	idx_course_published
created_at	timestamptz	NOT NULL		

示例记录：

```
{
  "id": "31c5e76a-0a1e-4f2a-8e9c-8b5a64d3cc30",
  "slug": "cpp-beginner-village",
  "title": "C++ 新手村：输入输出与变量",
  "difficulty": 1,
  "tags": ["IO", "变量", "基础语法"],
  "status": 2,
  "published_at": "2026-03-15T00:00:00Z"
}
```

lessons

字段	类型	约束	说明	索引建议
id	uuid	PK		
course_id	uuid	FK courses(id)		idx_lessons_course
slug	text	NOT NULL	章节 slug（course 内唯一）	UNIQUE(course_id, slug)
title	text	NOT NULL	标题	
order_no	int	NOT NULL	排序	idx_lessons_order
prerequisites	uuid[]		先修 lesson_id 列表	GIN(prerequisites)
est_minutes	int	NOT NULL	预计时长	
status	smallint	NOT NULL DEFAULT 1	1草稿2发布	idx_lessons_status

字段	类型	约束	说明	索引建议
current_version	int	NOT NULL DEFAULT 1	当前发布版本	
created_at	timestamptz	NOT NULL		

lesson_blocks

设计目标：可插入“文本、代码片段、互动测验、运行器、WASM 可视化”等，且可版本化。

字段	类型	约束	说明	索引建议
id	uuid	PK		
lesson_id	uuid	FK lessons(id)		idx_blocks_lesson
version	int	NOT NULL	对应 lessons.current_version	idx_blocks_version
block_type	text	NOT NULL	text/code/quiz/runner/wasm_demo	idx_blocks_type
content	jsonb	NOT NULL	内容块 payload	GIN(content jsonb_path_ops)
order_no	int	NOT NULL	顺序	idx_blocks_order
created_at	timestamptz	NOT NULL		

关于 jsonb 与索引：PostgreSQL 文档指出 jsonb 支持索引；同时 jsonb_path_ops 对 @>、@?、@@ 等操作符有性能优势。[18] GIN 索引适合做“内容包含/键存在/标签检索”等查询。[15]

示例 content（runner 块）：

```
{
 "language": "cpp",
 "starterCode": "#include <bits/stdc++.h>\nusing namespace std;\nint main(){\n  // TODO\n}\n",
 "stdinPreset": "5\n",
 "showExpected": false,
 "timeLimitMs": 2000,
 "memoryLimitKb": 131072
}
```

problems

字段	类型	约束	说明	索引建议
id	uuid	PK		
slug	text	UNIQUE NOT NULL		UNIQUE(slug)

字段	类型	约束	说明	索引建议
title	text	NOT NULL		idx_problem_title（全文检索可选）
statement	text	NOT NULL	题面（MVP 先用 text，后可 block）	
difficulty	smallint	NOT NULL	1–5	idx_problem_diff
tags	text[]		标签	GIN(tags)
starter_code	text	NOT NULL	C++ 模板	
checker_type	smallint	NOT NULL DEFAULT 1	1=stdout 比对 2=特殊判题 3=单元测试	idx_checker
judge_profile	jsonb	NOT NULL	限制/编译参数/运行方式	GIN(judge_profile jsonb_path_ops)
status	smallint	NOT NULL DEFAULT 2	发布状态	idx_problem_status
created_at	timestamptz	NOT NULL		

示例 judge_profile：

```
{
 "cpuTimeLimit": 2.0,
 "wallTimeLimit": 5.0,
 "memoryLimitKb": 128000,
 "maxProcesses": 60,
 "maxFileSizeKb": 1024,
 "enableNetwork": false
}
```

test_cases

字段	类型	约束	说明	索引建议
id	uuid	PK		
problem_id	uuid	FK problems(id)		idx_tc_problem
is_hidden	boolean	NOT NULL DEFAULT true	是否隐藏	idx_tc_hidden
group_no	int	NOT NULL DEFAULT 1	分组（部分分）	idx_tc_group
input_data	text	NOT NULL	stdin	
expected_output	text	NOT NULL	期望输出	
weight	int	NOT NULL DEFAULT 1	权重	
created_at	timestamptz	NOT NULL		

说明：若担心数据库泄露导致用例暴露，可把 input/expected 加密后存储或放对象存储（MVP 可先明文+隐藏用例不回传）。

submissions

字段	类型	约束	说明	索引建议
id	uuid	PK		
user_id	uuid	FK users(id)		idx_sub_user
submit_type	smallint	NOT NULL	1=run 2=judge	idx_sub_type
problem_id	uuid	FK problems(id)	nullable	idx_sub_problem
lesson_id	uuid	FK lessons(id)	nullable	idx_sub_lesson
language	text	NOT NULL DEFAULT 'cpp'	语言	
source_code	text	NOT NULL	源码	
stdin	text		运行输入	
judge0_token	text	UNIQUE	Judge0 submission token	UNIQUE(judge0_token)
status	smallint	NOT NULL DEFAULT 1	1排队2运行3完成4失败	idx_sub_status
created_at	timestamptz	NOT NULL		idx_sub_created
finished_at	timestamptz			idx_sub_finished

submission_results

字段	类型	约束	说明	索引建议
id	uuid	PK		
submission_id	uuid	FK submissions(id) UNIQUE	一对一	UNIQUE(submission_id)
compile_output	text		编译信息	
stdout	text		标准输出	
stderr	text		标准错误	
exit_code	int		退出码	
time_sec	double precision		用时	idx_time（可选）
memory_kb	int		内存	idx_memory（可选）
raw_result	jsonb	NOT NULL	Judge0 原始字段（脱敏）	GIN(raw_result)

字段	类型	约束	说明	索引建议
created_at	timestamptz	NOT NULL		

user_lesson_progress

字段	类型	约束	说明	索引建议
user_id	uuid	PK/FK users(id)		复合主键
lesson_id	uuid	PK/FK lessons(id)		复合主键
status	smallint	NOT NULL DEFAULT 1	1未开始2进行中3已完成	idx_prog_status
completion_rate	int	NOT NULL DEFAULT 0	0–100	
score	int	NOT NULL DEFAULT 0	小测/通关得分	
last_viewed_at	timestamptz			idx_prog_last

achievements / user_achievements / xp_logs

- achievements：徽章定义（slug、title、desc、rule_jsonb、xp_reward）
- user_achievements：用户获得徽章（user_id、achievement_id、earned_at）
- xp_logs：经验值流水（user_id、action、delta、ref_type/ref_id、created_at）

排行榜建议用 Redis Sorted Set（天然适配 leaderboard 与滑动窗口限流），Redis 官方文档把 Leaderboards 与 Rate limiters 都列为 Sorted Set 的典型用例。 [10]

audit_events

字段	类型	约束	说明	索引建议
id	uuid	PK		
actor_user_id	uuid	FK users(id)	操作人	idx_audit_actor
action	text	NOT NULL	如 admin.publish_course	idx_audit_action
target_type	text		courses/problems/...	
target_id	uuid			idx_audit_target
ip	inet			
payload	jsonb	NOT NULL	变更详情（脱敏）	GIN(payload)
created_at	timestamptz	NOT NULL		idx_audit_created

API 设计草图

总体风格与鉴权

- **REST 为主**（便于前端与后台快速协作），GraphQL 可作为可选的“聚合查询层”（例如首页/个人中心一次拉取多块数据）。
- 鉴权采用：Access Token（JWT，短 TTL）+ Refresh Token（httpOnly Cookie）。密码与登录策略参考 OWASP 认证与密码存储建议，避免明文/可逆加密存储。 [19]
- 对外 API 强制 HTTPS；对判题与后台接口施加更严格限流与审计。OWASP API Security Top 10 明确将“资源消耗/缺乏限流”作为关键风险点，建议对调用频率与资源做限制并向客户端返回限额与重置时间。 [20]

速率限制策略

推荐使用 Redis： - **通用 API**：滑动窗口计数（按 user_id / ip 分桶） - **登录接口**：更严格（例如 5 次/分钟/IP + 10 次/小时/user） - **判题提交**：按用户等级/订阅分层额度（例如免费用户 30 次/小时；付费 300 次/小时） Redis 文档指出 Sorted Set 可用于构建滑动窗口限流器；同时它也适合构建 leaderboard。 [21]

REST 端点列表（MVP）

模块	方法	端点	说明	鉴权	限流建议
Auth	POST	/api/v1/auth/register	注册	否	10/min/IP
Auth	POST	/api/v1/auth/login	登录	否	5/min/IP + 10/h/user
Auth	POST	/api/v1/auth/refresh	刷新 access token	是（refresh cookie）	30/min/user
Auth	POST	/api/v1/auth/logout	退出并吊销 refresh	是	60/min/user
User	GET	/api/v1/me	当前用户信息	是	120/min/user
Path	GET	/api/v1/paths	学习路径列表	否	300/min/IP
Path	GET	/api/v1/paths/:slug	路径详情（含节点与解锁状态）	是（可匿名返回灰度）	120/min/user
Course	GET	/api/v1/courses	课程列表	否	300/min/IP
Course	GET	/api/v1/courses/:slug	课程详情	否	300/min/IP
Lesson	GET	/api/v1/lessons/:id	章节详情（含 blocks）	是（记录进度）	120/min/user
Lesson	POST	/api/v1/lessons/:id/complete	标记完成/通关	是	60/min/user
Problem	GET	/api/v1/problems	题目列表（筛选/分页）	否	300/min/IP

模块	方法	端点	说明	鉴权	限流建议
Problem	GET	/api/v1/problems/:slug	题目详情（不含隐藏用例）	否	300/min/IP
Runner	POST	/api/v1/run	运行代码（不判题）	是	30/h/user（免费）
Judge	POST	/api/v1/submit	提交判题	是	30/h/user（免费）
Submission	GET	/api/v1/submissions/:id	查询提交详情	是	120/min/user
Submission	GET	/api/v1/submissions/:id/stream	SSE 推送状态	是	30 并发连接/user
Progress	GET	/api/v1/progress	学习进度汇总	是	60/min/user
Gamification	GET	/api/v1/leaderboards/xp	XP 排行榜	否	120/min/IP
Gamification	GET	/api/v1/achievements	徽章列表	否	120/min/IP
Admin	POST	/api/v1/admin/courses	新建/编辑课程	管理员	30/min/admin
Admin	POST	/api/v1/admin/problems	新建/编辑题目	管理员	30/min/admin
Admin	POST	/api/v1/admin/testcases/upload	上传用例	管理员	30/min/admin

请求/响应示例

注册：

```
POST /api/v1/auth/register
Content-Type: application/json

{
  "email": "allen@example.com",
  "password": "a-very-long-passphrase",
  "nickname": "Allen"
}
```

响应（示例）：

```
{
  "user": {
    "id": "8b2e8b9a-3c3a-4b0f-aac7-0b0f1bd8d4a1",
```

```
    "email": "allen@example.com",
    "nickname": "Allen"
  },
  "accessToken": "<jwt>"
}
```

提交判题（异步）：

```
POST /api/v1/submit
Authorization: Bearer <jwt>
Content-Type: application/json

{
  "problemSlug": "two-sum-cpp",
  "language": "cpp",
  "sourceCode": "#include <bits/stdc++.h>\nusing namespace std; int main(){...}",
  "clientTraceId": "0c9c8e..."
}
```

响应：

```
{
  "submissionId": "3d2d2f7f-12e1-45c2-8f77-3b9ab82d4b1a",
  "status": "QUEUED"
}
```

SSE 订阅状态（说明）：SSE 是服务器向客户端单向推送事件的机制，前端用 EventSource 接收；适合“提交状态更新”这种单向流。 12

```
GET /api/v1/submissions/3d2d2f7f-12e1-45c2-8f77-3b9ab82d4b1a/stream
Authorization: Bearer <jwt>
Accept: text/event-stream
```

事件示例：

```
event: status
data: {"status":"RUNNING","queuePosition":3}

event: done
data: {"status":"ACCEPTED","timeSec":0.12,"memoryKb":15360}
```

如后续要做双向互动（协作、实时教练提示），再引入 WebSocket：MDN 将 WebSocket 定义为创建与管理连接、在连接上收发数据的 Web API。 22

判题服务架构与部署方案

技术选型结论

MVP 推荐：**Judge0 CE 自托管**（独立于主站部署）。

依据： - Judge0 是开源在线代码执行系统，支持沙箱执行与可扩展架构；其特性包括“多文件程序/附加文件”“自定义编译参数、命令行参数、时间与内存限制”“详细执行结果”“Webhooks”等。[23] - Judge0 API 明确指出不推荐 wait=true（不利于扩展），因此平台应采用异步提交。[2] - Judge0 提供配置变量与上限（cpu_time_limit、wall_time_limit、memory_limit、max_processes_and_or_threads、max_file_size、enable_network 等）与限额（max_cpu_time_limit 等）机制，可直接用于“多租户不可信代码”风控。[24] - Judge0 部署流程官方建议基于 Docker 与 Docker Compose，并给出系统要求/部署步骤；其发布说明也记录了历史版本的安全漏洞修复（含 CVE），表明需要版本更新与安全治理流程。[25]

判题逻辑架构

推荐把“平台提交”分成两层：**平台提交编排层**（你自己写）+ **判题执行层**（Judge0）。

流程（异步）： 1. 前端调用 `/submit` → 主站写入 submissions（status=QUEUED） 2. 主站调用 Judge0 `POST /submissions?wait=false`，获取 `token` 存入 submissions.judge0_token（Judge0 文档示例展示提交返回 token；并明确 wait=true 不建议用于扩展）[2] 3. 主站将 submissionId 推入队列（Redis Streams / BullMQ / RabbitMQ 均可；MVP 可先 BullMQ+Redis） 4. Result Worker 拉取任务：轮询 Judge0 `GET /submissions/{token}` 直到完成，将结果写入 submission_results / submission_test_results 5. 主站通过 SSE 推送状态给前端（RUNNING / DONE）

Judge0 自身还提供 `/workers`、`/statistics`、`/health` 等健康与可观测接口（文档描述 /workers 会返回队列 size、worker idle/working、failed job 等），可用于自动扩容与告警。[24]

沙箱隔离与资源限制策略

资源限制（强制）

建议默认值（可按题目覆盖）： - CPU 时间：2s（通过题目 profile 写入） - 墙钟时间：5–10s（防止 sleep） - 内存：128–256MB - 最大进程/线程数：60（或更低） - 最大文件大小：1MB - 网络：默认禁用

这些限制字段与“可配置上限”在 Judge0 `config_info` 文档中给出名称、含义与默认值示例（如 cpu_time_limit、wall_time_limit、memory_limit、max_processes_and_or_threads、max_file_size、allow_enable_network、enable_network）。[24]

隔离（分层防御）

1. **执行容器隔离**：Judge0 依赖容器化部署（官方部署步骤基于 Docker/Docker Compose）。[25]
2. **Linux 内核隔离**：Judge0 的编译执行镜像被记录为使用 Isolate 更新；Isolate 项目明确说明其利用 Linux namespaces 与 cgroups 为不可信程序提供受限环境。[26]
3. **rootless 守护进程**：Docker rootless 模式允许以非 root 用户运行 Docker daemon 与容器，以缓解守护进程/运行时潜在漏洞带来的风险。[27]
4. **系统调用与访问控制**：使用 seccomp 限制容器内可用的内核系统调用；并使用 AppArmor profile 约束应用行为。Docker 官方文档分别说明了 seccomp 的用途与 AppArmor profile 的工作方式。[28]
5. **网络策略**：在 Kubernetes 部署中，用 NetworkPolicy 在 L3/L4 层控制 Pod 间及对外流量；K8s 文档明确 NetworkPolicy 可用于控制集群内与 Pod 到外界的流量（需要支持 NetworkPolicy 的 CNI）。[29]

日志与审计

最少应记录： - submission 的 user_id、ip、user_agent、时间、题目、语言、源码哈希（可用于查重/追踪） - Judge0 token、状态流转、资源使用、编译错误/运行错误摘要 - 管理员操作（题目/用例变更、发布、封禁用户）写入 audit_events

并保留“可追溯性”：当发生滥用或安全事件时，能回放“谁在何时提交了什么、系统怎么执行、用多少资源、结果是什么”。

部署方案与拓扑

MVP 部署（最省事但可控）

- **主站**：1 台 App VM（Nginx + API + 前端）
- **数据库**：建议托管 PostgreSQL（省去备份/高可用运维）；DO 定价可从 $15.15/月起 [30]
- **对象存储**：课程图片、用例包、备份包；Spaces 起步 $5/月（含一定存储与出网配额，视套餐） [31]
- **判题集群**：独立 1 台 Judge VM（仅暴露给主站内网/白名单），跑 Judge0（Docker Compose），并在配置上默认禁网与强资源限制

Judge0 官方发布说明给出了 Linux/Ubuntu 22.04 等要求、GRUB/cgroup 设置，以及 docker-compose 的启动步骤，可直接照做减少踩坑。 [25]

可扩展部署（有增长预期）

- 主站与 Worker Container 化上 Kubernetes
- 判题 Worker 独立 node pool（CPU 密集型）
- 使用 HPA 根据 CPU/自定义指标扩容；Kubernetes 文档说明 HPA 会自动更新工作负载副本数以匹配需求。 [32]
- 增加 NetworkPolicy、专用 VPC、只允许主站访问 Judge0 API

前端组件与页面原型结构

页面信息架构

页面	关键目标	必备组件
首页	展示学习路径入口与个人进度	路径卡片、继续学习、排行榜预览
学习地图（Path）	“闯关式”导航与解锁反馈	地图节点、锁/解锁态、奖励弹层
课程页（Course）	章节列表与进度	章节列表、完成度、开始按钮
章节页（Lesson）	边学边练（核心）	内容块渲染器、Monaco、运行面板、SSE 状态条
题库列表	按标签/难度练习	筛选器、分页、题目卡片
题目详情	编码提交与评测反馈	题面、样例、Monaco、提交按钮、测试结果面板
个人中心	成长与记录	XP/徽章、连续学习、提交历史
后台（Admin）	内容生产与发布	课程编辑器、题目/用例管理、发布与审计

核心前端组件清单

1. **CodeEditor**
2. 选型：Monaco Editor（VS Code 同款 Web 编辑器；官方标注不支持移动浏览器/移动 Web 框架） [1]
3. 降级策略：移动端 read-only 显示 + 引导到 PC；或改用轻量编辑器（可选）
4. **RunnerPanel**
5. Tab：stdout / stderr / compile output / memory/time
6. 状态：QUEUED/RUNNING/DONE/FAILED
7. 互动：复制错误、跳转到定位提示（后续）
8. **LessonBlockRenderer**
9. block_type = text / code / quiz / runner / wasm_demo
10. wasm_demo 需要 JS<->WASM 通道（见下）
11. **RealtimeStatusStream**
12. 默认 SSE：EventSource 订阅后端 stream；MDN 说明 SSE 客户端与 WebSocket 的接收处理类似，但 SSE 是单向连接。 [12]
13. WebSocket（可选）：用于未来的双向互动（协作/教练提示）；MDN 定义 WebSocket API 可用于建立与管理连接并收发数据。 [22]

WebAssembly 演示组件设计

建议做“少而精”的 2–3 个 WASM Demo，把 C++ 最劝退的点做成可视化：

- 栈/堆与对象生命周期（new/delete、RAII）
- 引用 vs 指针（地址、解引用、悬垂指针演示）
- vector 扩容与迭代器失效（进阶）

工具链： - Emscripten 官方说明其是面向 WebAssembly 的完整编译工具链，基于 LLVM，可把 C/C++ 编译到浏览器/Node/wasm runtime。 [33]
- Emscripten 文档说明其默认编译目标为 WebAssembly，并提供构建到 WebAssembly 的说明。 [34]
- JS 与编译产物交互可用 ccall/cwrap 或 Embind：官方文档分别描述了 ccall/cwrap 的调用方式与 Embind 绑定 C++ 类/函数以自然方式在 JS 中使用。 [35]
- emcc 是 Emscripten 的编译前端，行为上可作为 gcc/clang 的“替代入口”。 [36]

工程化与运维要点、里程碑、人力与成本、风险

CI/CD、监控、备份与运维

CI/CD

- 使用 GitHub Actions：GitHub 文档将 workflow 定义为由一个或多个 jobs 组成的可配置自动化流程，需要用 YAML 文件定义。 [37]
- 流水线建议：

- PR：lint/test/build + SAST（可选）+ 构建容器镜像（不推生产）
- main：构建镜像 → 部署 staging → e2e → 人工审批 → 部署生产
- 产物：前端静态资产（CDN）+ 后端镜像 + 迁移脚本

监控与告警（最小闭环）

- **指标**：Prometheus exposition format（文本格式）用于服务暴露 metrics；Prometheus 文档提供 exposition formats 与 instrumentation 实践建议（至少监控请求量、错误、延迟）。 [38]
- **告警**：Grafana Alerting 支持跨数据源创建查询与表达式并告警，是常见的指标/日志告警入口。 [39]
- **链路追踪/日志统一模型**：OpenTelemetry 是中立的可观测框架，用于采集与导出 traces/metrics/logs。 [40]

建议的关键指标： - 主站：HTTP 5xx、P95/P99 延迟、登录失败率、SSE 连接数、队列积压 - 判题：队列长度、worker idle/working、失败任务数（Judge0 /workers 可拿到关键维度） [24] - 数据库：连接数、慢查询、缓存命中、复制延迟（如有）

备份与恢复

- **逻辑备份**：pg_dump + pg_restore 提供灵活归档与选择性恢复；PostgreSQL 文档说明 pg_dump 输出可与 pg_restore 配合使用。 [41]
- **基础备份/PITR**：pg_basebackup 可对运行中的集群做基础备份，并用于时间点恢复或作为复制/standby 起点。 [42]
- 备份策略建议：
- 每日 pg_dump（保留 14 天）+ 每周全量（保留 8 周）
- 若业务进入增长期：开启 WAL 归档（PITR），每季度做一次恢复演练

Kubernetes 运维要点（进入规模化后）

- 探针：Kubernetes 文档说明 kubelet 使用 liveness/readiness/startup probes 来重启容器、控制流量接入与慢启动保护。 [43]
- 网络：NetworkPolicy 控制 Pod 间与对外流量。 [29]
- 弹性：HPA 自动扩容以匹配需求。 [32]

迭代发布里程碑与分周计划

假设团队：2 前端 + 2 后端 + 1 DevOps（兼职也可）+ 1 产品/教研 + 1 内容编辑（可由教研兼），目标 12 周出 MVP 上线。

分周交付表

周	关键交付物	负责人	验收点
W1	需求冻结、信息架构、MVP 列表；DB/接口草案	产品/后端	MVP 表与验收标准可执行；表结构评审通过
W2	Auth、用户中心基础；课程/章节读接口；前端框架搭建	FE/BE	登录与课程列表可跑通
W3	Lesson block 渲染器 v1；Monaco 接入；移动端降级	FE	PC 编辑器可用；移动端提示符合预期 [1]
W4	Judge0 自托管 PoC；提交 token 回写；SSE 状态推送	BE/DevOps	异步提交链路跑通；不使用 wait=true [2]

周	关键交付物	负责人	验收点
W5	题库与用例管理；判题结果落库；基础排行榜	BE	题目可提交并判定；榜单可查询 [44]
W6	后台管理 MVP；审计日志；限流策略上线	BE	Admin 可发布课程/题目；Redis 限流生效 [45]
W7	趣味化：WASM Demo 1（栈/堆）	FE	Emscripten 编译产物可交互 [46]
W8	趣味化：WASM Demo 2（指针/引用）+ Debug 题型	FE/教研	2 个 Demo 完整；新题型可出题
W9	数据看板与埋点；监控告警；备份策略落地	DevOps	Prometheus/Grafana/OTel 基线可用 [47]
W10	灰度发布（staging→prod）；安全加固（seccomp/AppArmor/NetworkPolicy）	DevOps	容器安全策略上线 [48]
W11	课程内容填充（20–30 节）+ 题库（100–150）	教研/内容	可完整跑一条学习路径
W12	上线与回归；运营首发活动；复盘与下阶段规划	全员	线上稳定；关键指标可观测；问题清单明确

mermaid 时间线图

```
timeline
  title C++ 趣味学习网站 0→1（12周）里程碑
  2026-03-09 : W1 需求冻结与MVP验收标准
  2026-03-16 : W2 Auth+课程基础接口
  2026-03-23 : W3 Lesson渲染器+Monaco接入
  2026-03-30 : W4 Judge0自托管PoC+异步提交+SSE
  2026-04-06 : W5 题库/用例/判题落库+排行榜
  2026-04-13 : W6 后台MVP+审计+限流上线
  2026-04-20 : W7 WASM Demo 1（栈/堆）
  2026-04-27 : W8 WASM Demo 2（指针/引用）+ Debug题型
  2026-05-04 : W9 监控告警+备份恢复演练
  2026-05-11 : W10 安全加固（seccomp/AppArmor/NetworkPolicy）
  2026-05-18 : W11 内容填充（课程+题库）
  2026-05-25 : W12 正式上线+复盘
```

安全与合规要点

沙箱与网络策略

- 默认禁网：Judge0 `enable_network` 等配置项可控制程序网络访问；应将默认设为 false，并禁止用户自行开启（通过 allow_enable_network 关闭或在平台侧不暴露）。 [49]
- 容器防护：rootless + seccomp + AppArmor 的组合为多层防御；Docker 文档对三者的目的与使用前提有明确描述。 [50]
- 集群网络：Kubernetes NetworkPolicy 控制 Pod 间/对外流量。 [29]

数据安全与隐私

- 密码与认证：遵循 OWASP Password Storage 与 Authentication Cheat Sheet 的核心建议（安全哈希、避免截断、避免弱策略）。[19]
- 敏感字段：refresh token、第三方密钥、判题 API key 均不落明文；对象存储启用最小权限与生命周期策略
- 隐私合规（原则性）：最小化收集（email、昵称即可），明确隐私政策；日志脱敏（源码可存 hash 或加密存储），对用户提供删除/导出机制（后续）

代码相似度检测建议

- Moss：Stanford 的 Moss 系统用于检测程序相似度，长期应用于编程课程抄袭检测。[51]
- JPlag：其 README 描述支持本地计算相似度，“不上传源码或结果到线上”，适合更严格隐私要求的场景。[52]
  MVP 建议做成**离线批处理**（夜间跑），输出“疑似对”供人工复核，避免误伤。

成本估算

以下以 DigitalOcean（USD/月）粗略估算三档，价格引用 DO 官方定价页：Basic Droplets（2GB=$12、4GB=$24、8GB=$48）、CPU-Optimized（4GB=$42、8GB=$84），托管 PostgreSQL（1GB=$15.15、2GB=$30.45），托管 Valkey/Redis（1GB=$15、2GB=$30），Spaces 起步 $5。[53]

> 假设：以“新加坡机房（SGP）就近”部署；出网主要靠套餐包含流量与 CDN（具体会随活动与内容体量变化）。如你选 AWS/GCP，成本结构类似但单价需按相应云重算。

档位	适用阶段与假设	资源组合（示例）	预计月成本（USD）
低	MVP 内测：DAU≤500，提交峰值≤5/s	App：Basic 4GB $24；Judge：CPU-Opt 4GB $42；托管 Postgres 1GB $15.15；Spaces $5；（Redis 先同机或后加托管）[54]	**约 $86 – $140**（视备份/日志/额外流量）
中	小规模增长：DAU≈2,000，峰值提交≈10/s	App：Basic 8GB $48；Judge：2×CPU-Opt 8GB = $168；托管 Postgres 2GB $30.45；托管 Valkey 1GB $15；Spaces $5 [54]	**约 $260 – $380**
高	稳定增长：DAU≈10,000，峰值提交≈30/s	App：2×Basic 8GB=$96；Judge：4×CPU-Opt 8GB=$336；托管 Postgres 8GB=$122.10；托管 Valkey 8GB=$120；Spaces ≥$20 [54]	**约 $690 – $1,000+**（日志/监控/备份与带宽会显著上升）

补充：若使用托管数据库，DigitalOcean 产品页强调托管数据库具备端到端安全能力，数据库在私网中、且数据传输与静态均加密（具体实现依厂商）。[55]

风险与缓解措施清单

风险	触发信号	影响	缓解措施（可执行）
判题安全事件（逃逸/挖矿/滥用）	CPU/带宽异常、失败率飙升、审计异常	高	禁网默认；rootless + seccomp + AppArmor；NetworkPolicy 阻断 egress；提交限流与配额；快速封禁与溯源 [56]
wait=true 导致扩展失败	长连接堆积、网关超时	中	坚持 wait=false 异步化；SSE 推送状态；队列化 worker 拉取结果 [2]
数据库性能退化（内容与进度增长）	慢查询、CPU 飙升	中	合理索引；jsonb 用 GIN/jsonb_path_ops；分区（提交表按月）与归档；读写分离（后期） [57]
内容生产不足（产品“空心化”）	完课率低、回访低	高	教研流水线：模板化出课；后台支持快速发布；每周固定产能；数据驱动迭代
抄袭与 AI 代写导致学习效果差	同题通过率异常，代码相似度异常	中	引入 Moss/JPlag 离线查重 + 人工复核；更多 debug/解释型题目与过程分；限制复制粘贴（可选） [13]
监控缺失导致线上事故扩大	发现晚、定位慢	中	Prometheus 指标、Grafana 告警、OTel 链路；定义 SLO/错误预算；演练故障与恢复 [47]
备份不可用（“有备份但恢复不了”）	恢复演练失败	高	pg_dump 日备 + pg_basebackup/PITR；每月做一次恢复演练并记录流程 [58]

[1] https://microsoft.github.io/monaco-editor/
https://microsoft.github.io/monaco-editor/

[2] [3] [7] [9] [24] [49] [56] https://ce.judge0.com/
https://ce.judge0.com/

[4] [6] [14] [18] https://postgresql.ac.cn/docs/current/datatype-json.html
https://postgresql.ac.cn/docs/current/datatype-json.html

[5] [37] https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax
https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[8] [17] [19] https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[10] [21] [44] [45] https://redis.io/docs/latest/develop/data-types/sorted-sets/index.html.md
https://redis.io/docs/latest/develop/data-types/sorted-sets/index.html.md

[11] [33] https://emscripten.org/
https://emscripten.org/

[12] https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events
https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

[13] [51] https://theory.stanford.edu/~aiken/moss/
https://theory.stanford.edu/~aiken/moss/

15 57 https://www.postgresql.org/docs/current/gin.html
https://www.postgresql.org/docs/current/gin.html

16 https://www.postgresql.org/docs/current/datatype-json.html
https://www.postgresql.org/docs/current/datatype-json.html

20 https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/
https://owasp.org/API-Security/editions/2023/en/0xa4-unrestricted-resource-consumption/

22 https://developer.mozilla.org/zh-CN/docs/Web/API/WebSocket
https://developer.mozilla.org/zh-CN/docs/Web/API/WebSocket

23 https://github.com/judge0/judge0
https://github.com/judge0/judge0

25 26 https://github.com/judge0/judge0/blob/master/CHANGELOG.md
https://github.com/judge0/judge0/blob/master/CHANGELOG.md

27 50 https://docs.docker.com/engine/security/rootless/
https://docs.docker.com/engine/security/rootless/

28 48 https://docs.docker.com/engine/security/seccomp/
https://docs.docker.com/engine/security/seccomp/

29 https://kubernetes.io/docs/concepts/services-networking/network-policies/
https://kubernetes.io/docs/concepts/services-networking/network-policies/

30 https://www.digitalocean.com/pricing/managed-databases
https://www.digitalocean.com/pricing/managed-databases

31 https://www.digitalocean.com/blog/understand-your-cloud-bill
https://www.digitalocean.com/blog/understand-your-cloud-bill

32 https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/
https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/

34 46 https://emscripten.org/docs/compiling/WebAssembly.html
https://emscripten.org/docs/compiling/WebAssembly.html

35 https://emscripten.org/docs/porting/connecting_cpp_and_javascript/Interacting-with-code.html
https://emscripten.org/docs/porting/connecting_cpp_and_javascript/Interacting-with-code.html

36 https://emscripten.org/docs/tools_reference/emcc.html
https://emscripten.org/docs/tools_reference/emcc.html

38 47 https://prometheus.io/docs/instrumenting/exposition_formats/
https://prometheus.io/docs/instrumenting/exposition_formats/

39 https://grafana.com/docs/grafana/latest/alerting/
https://grafana.com/docs/grafana/latest/alerting/

40 https://opentelemetry.io/docs/
https://opentelemetry.io/docs/

41 58 https://www.postgresql.org/docs/current/app-pgdump.html
https://www.postgresql.org/docs/current/app-pgdump.html

42 https://www.postgresql.org/docs/current/app-pgbasebackup.html
https://www.postgresql.org/docs/current/app-pgbasebackup.html

43 https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
https://kubernetes.io/zh-cn/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

52 https://github.com/jplag/JPlag
https://github.com/jplag/JPlag

53 54 https://www.digitalocean.com/pricing/droplets
https://www.digitalocean.com/pricing/droplets

55 https://www.digitalocean.com/products/managed-databases
https://www.digitalocean.com/products/managed-databases